7. Deep Learning — Why It Works

Source: Deep Learning: Foundations and Concepts, Chapter 1

What Is Deep Learning?

Deep learning = machine learning using **neural networks with many layers of learnable parameters**.

- A neural network learns a function from inputs to outputs by adjusting millions (or billions) of **weights** through training
- What makes it "deep": multiple **hidden layers** that learn increasingly abstract representations of the data
- The same fundamental framework applies to radically different problems: image classification, protein structure prediction, language generation, trading signals

> Trader translation: A deep learning model for predicting returns learns features like "short-term momentum pattern" in early layers, then "regime shifts" in later layers — automatically, without a human specifying these features.

The Tutorial Example: Polynomial Fitting

This example introduces key ML concepts using a simple curve-fitting problem.

Setup: 10 data points generated from sin(2πx) + noise. Fit polynomials of different orders M.

Order M	Behavior	Analogy
M=0 or 1	Underfit — too rigid to capture the curve	A linear model for a nonlinear market
M=3	Good fit — captures the structure	Right model complexity
M=9	Perfect training fit (E=0), wild oscillations, terrible test error	**Overfitting**

🎯 **Exam Flag: M=9 polynomial passes through all 10 training points exactly but generalizes terribly. This is the definition of overfitting — the model memorized noise.**

Key insight: When the polynomial order equals the number of data points (M=9 with N=10), the model has enough parameters to memorize training data but captures none of the underlying structure.

More data cures overfitting: With N=100 points, even M=9 gives a good fit. More data → can afford more complex models.

Error Functions and Training

The training objective is to minimize the **error function (loss)**:

```
E(w) = (1/2) × Σ [y(xₙ, w) - tₙ]²     (sum of squared errors)
```

- **w** = weights (parameters to be learned)
- **$y(x_n, w)$** = model prediction for input x_n
- **t_n** = true target value
- The factor 1/2 is for convenience (derivatives become cleaner)

For linear models, this has a closed-form solution. For deep neural networks, it must be solved iteratively using gradient descent.

Regularization (Weight Decay / L2 Penalty)

Problem: High-order models overfit by tuning weights to huge values to perfectly match training noise.

Solution: Add a penalty on large weights to the error function:

```
Ẽ(w) = (1/2) × Σ [y(xₙ,w) - tₙ]² + (λ/2) × ||w||²
```

Where `||w||² = w₀² + w₁² + ... + wₘ²`.

🎯 **Exam Flag: This is L2 regularization (also called Ridge or weight decay in neural networks). The λ parameter controls overfitting:**
- **λ too small** → overfitting (effectively no regularization)
- **λ too large** → underfitting (weights forced too close to zero)
- **λ just right** → controlled complexity, good generalization

The regularizer penalizes large coefficients, keeping them small and smooth — preventing the wild oscillations seen in unregularized high-order polynomials.

Hyperparameters and Model Selection

Hyperparameters = parameters set *before* training that control model complexity (not learned from data):
- **M** = polynomial order (or number of layers / hidden units in a neural network)
- **λ** = regularization strength

You cannot select hyperparameters by minimizing training error alone — that always leads to largest M and λ → 0 (maximum overfitting).

Solution: Validation set + Cross-validation

```
Training set → Fit model parameters (weights)
Validation set → Select hyperparameters (M, λ)
Test set → Final unbiased evaluation (touch ONCE)
```

K-fold cross-validation:

1. Split data into K equal groups
2. Train on K-1 folds, validate on 1 fold
3. Rotate K times, average scores
4. Best hyperparameter = lowest average validation error

🎯 **Exam Flag: Training set, validation set, test set are ALL different. Never optimize on the test set — it must remain untouched until final evaluation.**

Neural Networks: Architecture

A neural network processes data through layers of interconnected units (neurons):

```
Input → [Layer 1: Hidden] → [Layer 2: Hidden] → ... → Output
```

Single neuron computation:

```
a = Σ wᵢxᵢ            (pre-activation: weighted sum of inputs)
y = f(a)              (activation: nonlinear transformation)
```

Term	Meaning
x_i	Input values
w_i	Weights (strengths of connections, what we learn)
a	Pre-activation (linear combination)
$f(\cdot)$	Activation function (introduces nonlinearity)
y	Output of the neuron

Activation functions (critical for power of neural networks):
- **Step function (Perceptron)**: 0 if a ≤ 0, 1 if a > 0 — not differentiable, can't use gradient descent
- **Sigmoid**: smooth S-curve between 0 and 1
- **ReLU**: max(0, a) — most commonly used today
- **Tanh**: smooth curve between -1 and 1

🎯 **Exam Flag: Without nonlinear activation functions, any multi-layer neural network reduces to a single linear model — the depth provides no benefit.**

Brief History of Machine Learning

Era	Key Development
1960s	**Perceptrons** (Rosenblatt 1962) — single-layer networks with step function; guaranteed to converge for linearly separable problems; failed for XOR (Minsky 1969)
1980s	**Backpropagation** (Rumelhart et al. 1986) — efficient algorithm for computing gradients in multi-layer networks; enabled training of networks with hidden layers; differentiable activation functions; stochastic gradient descent
2012+	**Deep Learning revolution** — GPUs enabled training massive networks; AlexNet won ImageNet 2012; exponential growth in scale (GPT-4: ~1 trillion parameters)

Backpropagation

Information flows forward through the network (forward pass) to compute predictions. Errors flow backward (backward pass) to compute gradients.

```
Forward pass:  inputs → hidden layers → output → loss
Backward pass: loss → gradients flowing back through each layer
```

Each parameter's gradient tells us: "Which direction should I move this weight to decrease the loss?" Then: **Stochastic Gradient Descent (SGD)** updates weights one mini-batch at a time.

Representation Learning (Why Deep Networks Win)

Each hidden layer learns increasingly abstract **representations** of the input:

Layer	What it might learn (image example)
Layer 1	Edges and corners
Layer 2	Shapes and textures
Layer 3	Object parts
Layer 4	Full objects

For trading: Layer 1 might learn individual candlestick patterns, later layers learn multi-day regimes. This automatic feature extraction is why deep learning dominates when data is abundant.

Deep Learning Scaling Laws

Key insight of the deep learning era: **more data + more compute + bigger model = better performance**, often following power laws.

- From 2012 onward: compute used to train state-of-the-art models grew 10× per year
- Large Language Models (GPT, BERT, Claude): same architecture, different domains — one framework rules all

> **Transfer Learning:** Pre-train on massive general dataset → fine-tune on your specific task. Used when labeled data is scarce (e.g., a medical imaging model pre-trained on ImageNet).

Exam Cheat Sheet — Deep Learning Fundamentals

- **Neural network training**: minimize sum-of-squares loss (or cross-entropy for classification) via gradient descent
- **Weights**: learned parameters (like β in linear regression)
- **Hidden layers**: learn intermediate representations; more layers = deeper, more powerful
- **Activation function**: nonlinear function applied after each linear step; ReLU is most common
- **Backpropagation**: efficient algorithm for computing gradients through all layers
- **Overfitting**: model too complex for the data; solution = regularization (L2/weight decay) or more data
- **Regularization (L2 / weight decay)**: `loss + λ||w||²`; small λ = more flexible; large λ = simpler
- **Hyperparameters**: model complexity, λ — selected via validation set / cross-validation (NOT training set)
- **Train/validation/test split**: train on training set, tune hyperparameters on validation set, final evaluation on test set ONCE
- **Transfer learning**: pre-train on large dataset, fine-tune on target task

---

8. Regularization & Model Selection

Source: Introduction to Statistical Learning (ISL), Chapter 3.5 + Chapter 6 Supplement

3.5 — Parametric vs. Non-parametric Methods

Parametric Methods (e.g., Linear Regression)

- **Assume a functional form** for f(X): e.g., $Y = \beta_0 + \beta_1 X_1 + \ldots + \beta_p X_p$
- Only need to estimate a small number of coefficients
- Fast to train, easy to interpret, significance tests available
- **Weakness:** If the true relationship is not linear, the model fits poorly

Non-parametric Methods (e.g., kNN Regression)

- **No assumed functional form** — data shapes the fit directly
- kNN prediction: average of K nearest training points' responses
- More flexible, can fit any shape
- **Weakness:** requires much more data; degrades badly in high dimensions

---

When Does Linear Regression Beat kNN?

🎯 **Exam Flag: This comparison is directly exam-tested.**

Situation	Winner	Why
True relationship is **linear**	Linear Regression	LR directly fits the correct form; kNN must approximate it and incurs variance penalty
True relationship is **highly non-linear**	kNN	LR is systematically biased; kNN can adapt
High dimensionality (many features, p large)	Linear Regression	kNN suffers from the curse of dimensionality
Small dataset	Linear Regression	kNN needs enough neighbors to average; sparse neighborhoods → high variance

The Curse of Dimensionality

As the number of predictors p grows:
- kNN must search in higher-dimensional space
- Neighbors become farther away — the "nearest" points are no longer truly close
- Average over distant neighbors = high bias
- kNN performance degrades rapidly with p even if the true relationship is non-linear

🎯 **Exam Flag: kNN is competitive for p=1 or p=2 features, but linear regression often wins for p ≥ 4 even when the true relationship is non-linear.**

> Trader lesson: If you have 50 features (volume, momentum, various technical indicators), linear regression or tree-based methods will typically outperform kNN. kNN works when you have few, carefully engineered features.

Ridge Regression (L2 Regularization)

Problem: When the number of predictors p is close to n (or p > n), ordinary least squares (OLS) has very high variance — small changes in training data → wildly different coefficient estimates.

Ridge solution: Add an L2 penalty to the loss function:

```
Minimize: RSS + λ × Σβⱼ²
         = Σ(yᵢ - ŷᵢ)² + λ(β₁² + β₂² + ... + βₚ²)
```

🎯 **Exam Flag: λ controls the bias-variance tradeoff in Ridge:**
- **λ = 0**: Ridge = OLS (no regularization)
- **λ → ∞**: All β → 0 (null model, max bias, zero variance)
- **Optimal λ**: chosen via cross-validation

Effect of Ridge:
- All coefficients shrink towards zero but **never reach exactly zero**
- Ridge keeps ALL predictors in the model (no feature selection)
- Reduces variance at the cost of a small increase in bias
- Works best when OLS has high variance (p close to n, or highly correlated predictors)

Key requirement: Standardize predictors first (mean=0, std=1), because Ridge penalizes coefficient magnitude and scale matters.

Lasso (L1 Regularization)

Lasso (Least Absolute Shrinkage and Selection Operator) uses an L1 penalty:

```
Minimize: RSS + λ × Σ|βⱼ|
         = Σ(yᵢ - ŷᵢ)² + λ(|β₁| + |β₂| + ... + |βₚ|)
```

🎯 **Exam Flag: The key difference from Ridge — Lasso sets some coefficients to EXACTLY zero (performs automatic feature selection).**

Why does Lasso create zeros?
- Geometrically: the L1 constraint region is a diamond with corners at the axes
- The RSS contours often intersect the L1 diamond at a corner → sets that coefficient to 0
- The L2 circle (Ridge) has no corners → intersection is never exactly on an axis

Effect of Lasso:
- **Sparse models**: only a subset of predictors have non-zero coefficients
- **Automatic feature selection**: eliminates irrelevant features
- More interpretable than Ridge

Ridge vs. Lasso — Key Comparison

🎯 **Exam Flag: Memorize this table.**

	Ridge (L2)	Lasso (L1)
Penalty	$\lambda \times \Sigma\beta_j^2$	$\lambda \times \Sigma$
Zeros?	Never exactly 0	Yes — sparse
Feature selection?	No (keeps all)	Yes (eliminates features)
When it wins	Many small effects, correlated predictors	Few large effects, want interpretable sparse model
Interpretability	Moderate	High (fewer features)
Bias-variance	Both trade off as λ increases	Both trade off as λ increases

Both methods:
- Decrease variance and increase bias as λ increases
- λ selected via cross-validation
- Require standardized predictors
- Outperform OLS when p is close to n or predictors are correlated

---

Why Regularization Matters for ML4T

> **Finance-specific:** Factor models often have many correlated predictors (momentum, value, quality, etc.). Ridge handles multicollinearity well. Lasso picks the most important factors — useful for sparse factor model construction.

- **Portfolio signal generation:** Lasso naturally builds sparse factor models
- **Overfitting prevention:** Adding L2 penalty to neural networks = weight decay (Section 7)
- **Regime detection:** Regularized models are more stable across different market regimes

---

Exam Cheat Sheet — Regularization & Model Selection

- **Parametric (LR)**: assumes linear form; wins in high dimensions (p large) or when true relationship is linear

- **Non-parametric (kNN)**: no assumed form; wins for nonlinear data with few features; suffers from curse of dimensionality
- **kNN in high dimensions**: neighbors are far away → predictions become poor; linear regression often wins for $p \geq 4$
- **Ridge**: L2 penalty; shrinks all coefficients; none exactly zero; reduces variance; best when $p \approx n$ or multicollinear predictors
- **Lasso**: L1 penalty; sets some coefficients exactly to zero; automatic feature selection; better interpretability
- **λ selection**: always via cross-validation on validation set (never minimize training error for λ)
- **Standardize features**: required before applying Ridge or Lasso (penalty depends on scale)
- **Both Ridge and Lasso**: more biased than OLS but lower variance; net effect = lower test MSE when OLS overfits

---

9. Hedge Funds — Structure & Trading

Source: What Hedge Funds Really Do, Chapters 2 & 4

Fund Types Comparison

🎯 **Exam Flag: These fund facts appear frequently on exams.**

Feature	ETF	Mutual Fund	Hedge Fund
Regulation	SEC regulated	SEC regulated	Lightly regulated (US)
Trading	Throughout the day (like a stock)	Not traded throughout day (end-of-day NAV)	Not publicly traded
Fees (AUM)	~0.1–1.0%	~0.25–2.0%	**2% AUM + 20% profits** ("2 and 20")
Transparency	High	Moderate (must disclose holdings)	Very low (secretive)
Investors	Anyone	Anyone	Accredited investors only (wealthy institutions/individuals)
Advertising	Allowed	Allowed	Prohibited (historically)
Leverage	Limited	Limited	Extensive

🎯 **Exam Flag: Hedge funds charge "2 and 20" — 2% of AUM regardless of performance + 20% of profits.**
- Example: Portfolio earns 10%, fund charges 2% + (0.2 × (10% - 2%)) = 2% + 1.6% = 3.6% total fees
- This makes hedge fund managers wealthy even in flat years

Hedge Fund Structure

Who can invest: "Accredited investors" — upper-tier wealth (replaces formal regulation with investor sophistication)

Why secretive: Cannot legally protect intellectual property (financial strategies), so secrecy is the only way to prevent competitors from copying strategies.

Leverage: Hedge funds borrow extensively to amplify returns. Example: **130/30 strategy**:
- Long 130% of capital (borrow 30% extra) — bet on stocks rising
- Short 30% of capital — bet on stocks falling
- Net: 100% exposure but with more alpha generation

Strategies (4 main types):
1. **Equity (Long/Short)**: Buy undervalued stocks, short overvalued stocks — the original

"hedged" approach
2. **Arbitrage**: Exploit price discrepancies for identical/related assets
3. **Momentum/Direction**: Bet on macro price direction in a market
4. **Event-driven**: Trades triggered by specific events (mergers, supply disruptions, corporate actions)

Performance Evaluation

Return Calculation

Daily return:

```
Return = [Value(t) / Value(t-1)] - 1
```

Annualized return (252 trading days):

```
annual_return = cumprod(daily_returns + 1) - 1
```

CAGR (Compound Annual Growth Rate):

```
Ending Value = Starting Value × (1 + CAGR)^n
```

Where n = number of years. A 7.2% CAGR doubles a portfolio in ~10 years.

> **Rule of 72:** Divide 72 by the annual growth rate to estimate how many years to double. 8% CAGR → 72/8 = 9 years to double.

Sharpe Ratio

🎯 **Exam Flag: The Sharpe ratio formula is directly tested.**

```
Sharpe Ratio = (r_portfolio - r_risk_free) / std(r_portfolio)
```

- r_portfolio = portfolio return
- r_risk_free = Treasury bill rate (the "free" baseline)
- std(r_portfolio) = standard deviation of portfolio returns (risk)

Interpretation:
- S&P 500 long-term Sharpe ≈ 0.5
- Sharpe ≥ 1.0: very good (rarely achieved sustainably)
- Higher is always better — more return per unit of risk

🎯 **Exam Flag: In Python, annualized Sharpe ratio from daily returns:**

```
sharpe = np.sqrt(252) * daily_returns.mean() / daily_returns.std()
```

Risk Measures

Measure	What It Is	Notes
Standard Deviation	Volatility of returns	Most common; treats up and down equally
Drawdown	Max drop from peak to trough	Hedge funds care deeply about this
Sortino Ratio	Like Sharpe but only penalizes downside	Better than Sharpe for asymmetric strategies

Benchmark comparison: Active managers are compared to benchmarks (S&P 500, HFRX hedge fund index). Excess return above benchmark = **alpha**.

Market-Making Mechanics (Chapter 4)

Market Structure

NYSE (New York Stock Exchange): Physical floor, market makers called **specialists**.
NASDAQ: Fully electronic, no floor — market makers coordinate electronically.

Investors → Brokerage firms → Specialists/Market makers → Exchange

Bid-Ask Spread

🎯 **Exam Flag: Know the order book structure.**

Term	Definition
Ask price	Price at which sellers offer to sell
Bid price	Price at which buyers offer to buy
Market spread	Ask - Bid = profit for market makers

- Highly liquid markets → many market makers → intense competition → narrow spreads
- Illiquid/niche markets → few participants → wide spreads
- **Widening spreads** = early warning sign of liquidity crisis (e.g., 2008 fixed income crisis)

Example: IBM quoted at $200.50 ask / $199.75 bid → spread = $0.75 per share

Order Types

Order Type	Behavior
Market order	Execute immediately at current market price
Limit order	Execute only at specified price or better (buy ≤ limit, sell ≥ limit)
Stop loss	Automatically sell if price falls X% below purchase price (risk management)
Trailing stop	Stop loss that adjusts to the highest recent price (preserves gains)

Short Selling

🎯 **Exam Flag: Know the short selling mechanics.**

Short selling = **betting that a stock's price will fall**:

```
1. "Sell to open": Borrow shares from a holder and sell them (receive cash)
2. (Wait for price to fall)
3. "Buy to close": Repurchase shares at lower price and return to lender
4. Profit = Sale price - Repurchase price - Rebate (interest to share lender)
```

Risks of shorting:
- **Unlimited downside**: Stock price can rise without bound → unlimited losses
- **Maximum long loss = 100%** (stock can't go below 0); maximum short loss = ∞
- Company executives may retaliate against short sellers (restrict analyst access)
- Long-term trend of stocks is upward → shorting fights the trend

The Order Book

The order book is a real-time list of all outstanding buy and sell orders at each price level:

```
Sell (Ask) side:     Buy (Bid) side:
100 shares @ $100.10  300 shares @ $99.95
100 shares @ $100.05  200 shares @ $99.90
100 shares @ $100.00  300 shares @ $99.85

Spread = $100.00 - $99.95 = $0.05
```

Reading the order book:
- More shares on ask (sell) side → likely price decline
- More shares on bid (buy) side → likely price rise
- Market order fills by walking through the book (best bid/ask first)

High-Frequency Trading (HFT)

Competitive advantage = milliseconds:
- Computers execute trades far faster than humans
- Firms invest in fiber optic cables, co-location at exchanges (physically close to matching engine)
- **Dark pools**: informal exchanges between brokerage firms to clear trades internally, avoiding public exchange fees

Front running: A broker trading ahead of clients using knowledge of pending orders — illegal but historically common.

Arbitrage Strategies

Pure arbitrage: Buy cheap in one market, sell expensive in another simultaneously (risk-free profit).

Statistical arbitrage: Exploit temporary price divergence from estimated true value:
- Market price > intrinsic value → short opportunity
- Market price < intrinsic value → long opportunity
- Efficient markets quickly arbitrage these away

Regression to the mean: A volatile asset driven from true value tends to return to it — basis for mean-reversion strategies.

Value investing: Warren Buffett style — buy stocks at a "margin of safety" below estimated intrinsic value, wait for market to recognize the value.

Exam Cheat Sheet — Hedge Fund Basics

- **Hedge Fund fees**: 2% AUM + 20% profits ("2 and 20")
- **Mutual fund fees**: 0.25–2.0% AUM, no performance fee
- **ETF**: traded like a stock throughout the day; lowest fees (~0.1–1%)
- **Hedge fund investors**: accredited investors only (wealthy, institutional)
- **Sharpe Ratio**: (return - risk-free rate) / std of returns; annualized in Python: `sqrt(252) × mean/std`
- **Drawdown**: max % decline from peak to trough
- **Short selling**: sell borrowed shares ("sell to open"), buy back later ("buy to close"); unlimited downside risk
- **Bid**: price buyer will pay; **Ask**: price seller will accept; **Spread = Ask - Bid**
- **Market order**: execute immediately; **Limit order**: execute at specified price or better
- **Order book**: real-time list of all bids and asks; more asks = price likely falls
- **Long/short equity**: the original "hedge" — buy undervalued, short overvalued stocks

---

10. AI in Trade Execution & Factor Investing

Source: Handbook of AI and Big Data Applications in Investments, Chapter 7

Overview: ML in Practice for Trading

Despite enthusiasm, adoption of ML on the buy side (asset managers) remains limited — surveys show only ~22-28% of buy-side firms regularly use ML in decision-making. The key challenges:
- **Data quality**: Raw trading data is noisy, inconsistently tagged, and requires significant cleaning
- **Explainability**: Neural networks are "black boxes" — trading firms need explainable models
- **Labeling**: Much trading data is unlabeled or has free-form text fields
- **Scale**: Firms need efficient processes to run ML-based analytics at scale

> **Practical rule**: Before any ML model, data cleansing, normalization, and outlier removal typically consume the most time (80%+).

Transaction Cost Analysis (TCA)

TCA = systematic post-trade analysis to measure and improve execution quality.

Traditional TCA: Observational — break down trade costs by factors like liquidity, volatility, spread.

ML-enhanced TCA: Uncover subtle patterns in trading data that traditional analysis misses.

🎯 **Feature importance in TCA (Use Case 1):**

Feature importance from a Random Forest model on trading cost data revealed:
1. **Trade Horizon** (0.29) — most important: how long the order took to execute
2. **Traded Value** (0.17) — total notional traded
3. **Historical Volatility** (0.17) — stock's historical price variability
4. **Participation Rate** (0.12) — % of market volume the order represented
5. Percent of average daily volume, spread, market cap, sector (smaller importance)

Three methods for computing feature importance:

Method	How	Pros/Cons
Built-in (Random Forest)	Gini impurity decrease per split	Fast; native to trees
Drop-column	Remove one feature, measure accuracy drop	Accurate but expensive (retrain each time)
SHAP values	Game theory — how much each feature contributes	Most informative; computationally expensive

---

Learning Types Applied in Trading

Use Case 2: Semi-Supervised Learning for Data Tagging

Problem: Trading datasets often have only a small fraction of labeled examples (e.g., commission type tags) with vast amounts of unlabeled data.

Semi-supervised learning = train on the small labeled portion, then extend predictions to the unlabeled majority.

Process:
1. Parse broker codes to extract tags (e.g., "brokerA_HT" → "Not Execution Only")
2. Train classification model on labeled portion
3. Predict labels for unlabeled portion
4. Validate: does the unsupervised clustering match?

Key insight: For mixed numeric + categorical data in trading:
- K-means: works on numerical features
- K-modes: works on categorical features
- K-prototype (Huang 1998): combines both

Use Case 3: NLP for Trading Instructions

Problem: Portfolio managers send trading instructions as free-form text:
- "Target this order for the close"
- "Part of a prog, trade cash neutral"
- "Avoid impact, trade in line with volume"

Solution: NLP to extract structured tags from text → enables automation of instruction parsing and systematic compliance checking.

Algo Wheels and Model-Assisted Execution

Use Case 4: Algo Wheel (A/B Testing for Trading Algorithms)

An **algo wheel** is an automated, randomized routing system that:
1. Accepts an order

2. Routes it randomly to one of several pre-defined broker algorithms
3. Tracks post-trade outcomes to measure each broker's performance
4. Adjusts future routing allocations based on performance

This is essentially **A/B testing** applied to trading algorithms:
- Each order is an experiment
- Need ~300 orders per broker to build a useful dataset
- ML models (Random Forest) predict market impact to **normalize** order difficulty across brokers

Endogeneity risk: Order horizon (how long execution takes) improved model accuracy but was *controlled by the broker* — including it would let brokers game the model. Always check for endogeneity in feature selection.

Use Case 5: Trading Strategy Recommendation

The full pipeline:
1. Feature importance identifies key inputs (order attributes, market conditions, NLP-derived features)
2. Random forest models trained for each algorithm style (VWAP, implementation shortfall, liquidity seeking)
3. Model recommends optimal algorithm for each order
4. Trader can **override** the recommendation — and the override itself is logged for future model improvement

Algorithm trading styles:

Style	When to Use
VWAP (Volume-Weighted Average Price)	Minimize market impact for large orders; execute in proportion to market volume
Implementation Shortfall	Minimize slippage from decision price to execution price
Liquidity Seeking	Find dark pools and alternative venues for large blocks
Dark	Route entirely through dark pools (no price impact on lit market)

Key Lessons from AI in Practice

🎯 **Exam Flag: These reflect real-world ML pitfalls that appear in exam questions.**

1. **Data quality first**: All ML analyses begin with data, but "not all data are created equal" — noise, missing tags, free-form text all require preprocessing
2. **Model transparency matters**: In trading, black-box neural networks are often rejected in favor of explainable models (Random Forest, Linear Regression) — even if slightly less accurate
3. **Peer review everything**: Have non-model builders challenge your model's data, features, and assumptions

4. **Start simple**: Random Forest often outperforms more complex alternatives on financial datasets with mixed numeric/categorical features
5. **Track overrides**: When a human overrides an ML recommendation, capture and analyze why — invaluable training signal

---

Connecting to ML4T Exam Topics

🎯 **Exam-relevant connections from this chapter:**

- **Feature Importance** → directly relates to: which predictors matter in a factor model
- **Random Forest for TCA** → connects to Section 6 (Random Forest: reduces variance, handles mixed data)
- **Semi-supervised learning** → labeled + unlabeled data: practical when full labels are expensive
- **Model explainability** → connects to Section 7 (DL: black box vs. white box trade-off)
- **Data leakage risk** → endogenous features (order horizon controlled by broker = data leakage)

Exam Cheat Sheet — AI in Trading

- **TCA** = post-trade analysis to measure execution quality vs. benchmark (VWAP, implementation shortfall)
- **Feature importance**: Random Forest (built-in), drop-column, or SHAP values
- **Semi-supervised learning**: small labeled data + large unlabeled → combine supervised + unsupervised
- **Algo wheel**: randomized A/B testing of broker algorithms to measure performance fairly
- **VWAP**: execute in proportion to market volume → minimize impact
- **Implementation shortfall**: minimize difference between decision price and average fill price
- **Dark pools**: informal exchanges with no public price impact; used for large block trades
- **Data quality/endogeneity**: always check that features aren't controlled by the party being evaluated

---

11. Hedge Fund Strategies — Valuation & CAPM

Source: What Hedge Funds Really Do, Chapters 5 & 7

Chapter 5: Company Valuation

Hedge funds profit by identifying when a stock's **market price diverges from its true value**:
- Price < true value → **Long** (buy the stock, profit when it rises to fair value)
- Price > true value → **Short** (sell the stock, profit when it falls)

This is only actionable if you can independently estimate true value. Three methods:

Method 1: Book Value (Asset-Based Valuation)

```
Book Value = Total Assets - Total Liabilities = Stockholder's Equity
```

Price-to-Book Ratio:

```
P/B = Share Price / Book Value per Share
```

- P/B < 1.0: Stock trading below book value → potential value opportunity (or distress signal)
- P/B >> 1.0: Market prices in significant growth expectations

Limitations: Book value uses historical cost (not current value) — tends to understate true value. In bankruptcies, book value can overstate (forced asset sales yield less than book).

Method 2: Intrinsic Value (Dividend-Based / DCF)

Time Value of Money:

> Money today is worth more than money in the future because today's money can be invested elsewhere.

Discounting future cash flows:

```
Present Value = Future Value / (1 + Discount Rate)^n
```

Where n = number of years. The **discount rate** is the opportunity cost — the return you could earn on an alternative investment of equal risk.

Gordon Growth Model (stock with perpetual dividends):

```
Intrinsic Value = Dividend / (Discount Rate - Growth Rate)
```

Example: Stock pays $2/year dividend, discount rate = 10%, expected growth rate = 5%:

```
Intrinsic Value = $2 / (0.10 - 0.05) = $2 / 0.05 = $40
```

Method 3: Earnings-Based Valuation

P/E Ratio (Price-to-Earnings):

```
P/E = Share Price / Earnings per Share
```

- Compare P/E to industry average or historical P/E
- P/E below historical average → potential undervaluation
- High P/E → market expects high future growth (or the stock is overvalued)

Earnings growth projection: Project future earnings, apply a P/E multiple, discount back to present.

> **Fundamental Analysis** (book value, DCF, earnings) is distinct from **Technical Analysis** (uses only historical price/volume data). EMH's weak form says technical analysis can't work; semi-strong form says fundamental analysis also won't work if prices already reflect all public info.

Chapter 7: CAPM (Capital Asset Pricing Model)

🎯 **Exam Flag: CAPM is one of the most tested topics. Know alpha, beta, and how they're used.**

The Core Idea

Every stock return = two parts:
1. **Market-driven return** (systematic risk — can't diversify away)
2. **Company-specific return** (idiosyncratic risk — can diversify away)

CAPM equation:

```
Return(stock) = Alpha + Beta × Return(market) + ε
```

This is literally a simple linear regression of stock returns on market returns.

Alpha (α)

🎯 **Exam Flag:**

Alpha	Meaning
α > 0	Stock outperforms the market on a risk-adjusted basis = investing skill
α = 0	Stock performs in line with what market risk predicts
α < 0	Stock underperforms market on risk-adjusted basis

Alpha = Y-intercept of the regression line (stock return vs. market return scatter)

> On average across all stocks, alpha = 0 (because all stocks together = the market). Any stock with positive alpha is offset by another with negative alpha.

"**Seeking alpha**" = finding stocks with genuine excess return above what their market risk would predict. This is the core goal of active management.

Beta (β)

🎯 **Exam Flag:**

Beta	Meaning
β = 1.0	Moves exactly with the market
β > 1.0	More volatile than market (amplified swings) — speculative stocks
β < 1.0	Less volatile than market — stable large companies
β = 0	No correlation to market
β < 0	Moves opposite the market

Beta = slope of the regression line (stock return vs. market return)

Examples:
- ExxonMobil (XOM): β ≈ 0.77 (large, stable company, slightly less volatile than market)
- Small speculative biotech: β often 1.5–3.0
- Gold ETF: sometimes negative β (safe-haven asset)

"**Buying beta**" = investing in high-beta stocks for amplified market exposure. Higher expected return but also higher risk.

Correlation

Correlation coefficient (−1 to +1) measures how much two assets move together:
- **+1.0**: Perfect positive correlation (always move together)
- **0**: No relationship
- **−1.0**: Perfect negative correlation (always move oppositely)

🎯 **Exam Flag: Combining assets with low or negative correlation reduces portfolio volatility.**

Correlation is the key to diversification benefits — a portfolio of negatively correlated assets has lower volatility than any single asset.

Portfolio Return in CAPM

```
Return(portfolio) = Σ w_i × Return(stock_i)
                  = Σ w_i × [Alpha_i + Beta_i × Return(market)]
```

Where w_i = weight of stock i in portfolio.

Portfolio Beta = weighted average of individual betas:

```
Beta(portfolio) = Σ w_i × Beta_i
```

Beta-Balanced (Market-Neutral) Hedge Fund Strategy

🎯 **Exam Flag: The beta-neutral/market-neutral strategy is a core hedge fund approach.**

Goal: Portfolio with Beta = 0 (no net market exposure)

```
Σ w_i × Beta_i = 0
```

A manager who is long XOM (β=0.77) and short SPY (β=1.0) in a 50/50 dollar-equal split:
- Long 50% × 0.77 = +0.385 beta
- Short 50% × 1.0 = −0.50 beta
- Net beta = −0.115 (slightly net short)

To truly balance, hold 56% XOM long and −44% SPY short.

Why beta-neutral?
- Market exposure provides no alpha (you can get that cheaply with an ETF)
- A beta-neutral fund's returns are driven entirely by stock selection skill (alpha)
- If the market crashes, the short positions offset the long losses

Leverage amplifies alpha:
- Excess returns from stock selection are small (fractions of a percent per trade)
- Hedge funds leverage 20-30x to amplify these small alphas into meaningful returns

---

Exam Cheat Sheet — Valuation & CAPM

- **Book value** = assets − liabilities; P/B < 1 often signals undervaluation
- **DCF/Intrinsic value**: discount future dividends/cash flows at opportunity cost rate
- **P/E ratio**: price / EPS; compare to peers and historical values
- **Alpha**: Y-intercept of stock return vs. market return regression; positive = outperformance
- **Beta**: slope of that regression; measures market sensitivity
- β > 1 = more volatile than market; β < 1 = less volatile; β = 0 = no correlation
- **CAPM**: Return(stock) = Alpha + Beta × Return(market)
- **Portfolio return**: weighted sum of individual returns
- **Portfolio beta**: weighted average of individual betas

- **Beta-neutral portfolio**: $\Sigma(w_i \times \beta_i) = 0$; returns driven by alpha only
- **Buying beta**: invest in high-beta stocks for amplified market exposure (cheap, no skill needed)
- **Seeking alpha**: finding stocks that systematically outperform — the goal of active management

---

12. Portfolio Management, EMH & Data Quirks

Source: What Hedge Funds Really Do, Chapters 8, 9, 10 & 12

Chapter 8: The Efficient Market Hypothesis (EMH)

🎯 **Exam Flag: EMH and its three forms are heavily tested. Memorize these.**

EMH = the theory that asset prices reflect all available information instantly, leaving no exploitable opportunities for abnormal returns.

Three Forms of EMH

Form	What Prices Reflect	What CANNOT Work
Weak Form	All historical price and volume data	Technical analysis (chartism)
Semi-Strong Form	All publicly available information (financials, news, filings)	Technical analysis + Fundamental analysis
Strong Form	ALL information, including inside information	Technical analysis + Fundamental analysis + Insider trading

If weak form is correct:
- Technical analysis (using price history alone) cannot generate excess returns
- Fundamental analysis CAN still work — original research can create information advantage

If semi-strong form is correct:
- Both technical and fundamental analysis fail (information already in prices)
- Only private/illegal information provides an edge → most hedge fund criminality involves this

If strong form is correct:
- Nothing works — even insiders can't beat the market
- Most evidence: strong form does NOT hold (insider trading is profitable before detection)

Reality: Most evidence supports weak or semi-strong form for large-cap US stocks (NYSE). Less efficient for small-cap, emerging markets, and complex instruments.

> **Paradox:** Active managers (like hedge funds) may be precisely the agents who *make* markets efficient — by immediately trading on any discrepancy, they eliminate it. Hedge funds contribute to efficiency by competing it away.

Chapter 9: The Fundamental Law of Active Portfolio Management

🎯 **Exam Flag: IR = IC × √BR is one of the most tested formulas. Memorize it cold.**

The Formula

```
IR = IC × √BR
```

Term	Definition
IR (Information Ratio)	Risk-adjusted return: Mean(alpha) / Std(alpha) — like Sharpe ratio but for active returns only
IC (Information Coefficient)	Correlation of manager's predictions with actual outcomes; measures **skill**
BR (Breadth)	Number of independent investment decisions per year; measures **diversification/opportunity**

Intuition: The Coin Flip Example

- **IC** = edge on each flip (51% heads instead of 50%)
- **BR** = number of flips
- With $1,000 total:
- One $1,000 bet: Expected return = $20, Std Dev = $31.62, Sharpe = 0.63
- 1,000 × $1 bets: Expected return = $20, Std Dev = $1, Sharpe = **20.0**
- More bets with same skill = dramatically better risk-adjusted return

🎯 **Exam Flag: To DOUBLE performance at the same IC, you need FOUR TIMES as many trades (because √4 = 2).**

IC: Measuring Skill

- IC = correlation between predictions and actual outcomes
- IC = 1.0: perfect predictor
- IC = 0: random (no skill)
- IC = -1.0: always wrong (contrarian)

Real-world IC for top managers: typically 0.05–0.15 (modest but consistent skill)

BR: Measuring Breadth

Two ways to increase breadth:
1. **Hold more positions** (diversification) — Warren Buffett vs. RenTec comparison
2. **Trade more frequently** (higher turnover) — quantitative/high-frequency trading

🎯 **Exam Flag: Warren Buffett (concentrated, high IC) vs. Renaissance Technologies (broad, moderate IC) — both successful for different reasons:**
- Buffett: Very high IC, narrow breadth → focuses on circle of competence
- RenTec: Modest IC, enormous breadth → computers trade thousands of stocks many times per day

Systematic Risk vs. Specific Risk

- **Systematic risk**: Market-wide risk that affects all stocks (can't diversify away)
- **Specific risk**: Company-specific risk (can be reduced through diversification)

Diversification reduces specific risk but NOT systematic risk. Adding stocks beyond ~20-40 provides diminishing additional risk reduction.

Chapter 10: Modern Portfolio Theory (MPT) & The Efficient Frontier

🎯 **Exam Flag: The efficient frontier, negative correlation benefit, and portfolio optimization are tested.**

The Efficient Frontier

Markowitz (1952): Portfolio construction can achieve higher risk-adjusted return than any individual asset by combining assets with low/negative correlations.

- Plot each asset as a point in (risk, return) space
- **Efficient Frontier** = the curve connecting all portfolios that offer the maximum return for a given risk level
- Portfolios below the efficient frontier are suboptimal — same risk, lower return

```
Goal of portfolio optimization: Find portfolio weights that minimize risk
for a target return (or maximize Sharpe ratio)
```

Why Low/Negative Correlation Reduces Risk

Example: Stocks ABC and DEF are highly correlated ($\rho = 0.9$) — buying both gives little diversification benefit. Stock GHI is negatively correlated ($\rho = -0.9$) with ABC.

A portfolio of 25% ABC + 25% DEF + 50% GHI:
- Total return = weighted average (unchanged)
- Volatility = **much lower** than any individual stock

🎯 **Exam Flag: Combining negatively correlated assets reduces portfolio volatility without reducing expected return. This is the core magic of diversification.**

Optimization Components

Component	Portfolio Context
Decision variables	Portfolio weights w_i (what % in each stock)
Objective function	Minimize variance OR maximize Sharpe ratio
Constraints	$\Sigma w_i = 1$ (all money invested); $w_i \geq 0$ (no shorts for long-only); max weight per position

Sharpe ratio on scatter plot = slope of ray from origin to portfolio point (steeper = higher Sharpe)

The optimal portfolio = portfolio on efficient frontier where Sharpe is maximized (tangency portfolio).

Chapter 12: Overcoming Data Quirks

🎯 **Exam Flag: These Python/data concepts appear on every exam. Know them exactly.**

Adjusted vs. Actual Stock Prices

Problem: Stock prices change not just from supply/demand, but also from corporate actions:
- **Stock splits** (e.g., 2-for-1 split: shares double, price halves — total value unchanged)
- **Dividends** (company distributes cash → share price drops by dividend amount)

Solution: Use adjusted (split/dividend-adjusted) prices for backtesting.

🎯 **Exam Flag:**
- **Actual close price** = unadjusted raw price — shows a 50% price drop on a 2-for-1 split day
- **Adjusted close price** = retroactively adjusts all historical prices to account for splits and dividends
- Always use adjusted prices for return calculations and strategy backtesting!

Stock Splits

- 2-for-1 split: 100 shares × $100 → 200 shares × $50 (total value unchanged)
- All historical actual prices before the split date are halved in adjusted data
- IBM example: actual price shows $300 → $75 drop in 1979 (4-for-1 split) — NOT a loss; adjusted prices show continuous trend

Dividends

- Board declares dividend → stock price drops by ~dividend amount on ex-dividend date
- Adjusted price retroactively adjusts past prices downward
- Effect: backtesting captures the dividend "return" correctly

Missing Data / Breaks in Series

🎯 **Exam Flag: Fill forward first, fill backward where you cannot fill forward.**

```
Rule: "Fill forward first, then fill backward"
```

- **Fill forward** (ffill): Use the last known value for missing periods
- Example: NASDAQ halted 12:14–3:10 PM → use 12:13 PM value for all gaps
- **Fill backward** (bfill): Use the first known value for missing periods at the beginning

Survivorship Bias

🎯 **Exam Flag: Survivorship bias is a major pitfall in backtesting.**

Survivorship bias: Using only stocks that survived to the end of your study period — companies that went bankrupt, were acquired, or were delisted are missing.

Effects on backtesting:
- Artificially inflates measured returns (failed companies removed from sample)
- Strategies appear to work better than they actually do in real trading
- ~70% of hedge funds from 10 years ago no longer exist

Fix: Use point-in-time data that includes delisted/failed companies.

Exam Cheat Sheet — Portfolio Management & Risk

🎯 **Critical formulas and facts:**

- **EMH Weak form**: historical prices/volume can't predict future → technical analysis fails
- **EMH Semi-strong**: all public info in prices → fundamental analysis also fails
- **EMH Strong**: all info in prices → even insiders can't beat market
- **IR = IC × √BR**: Information Ratio = skill × √breadth; doubling IR requires 4× breadth at same skill
- **IC**: correlation of predictions with outcomes; perfect predictor = 1.0
- **Efficient Frontier**: set of portfolios with max return for given risk; optimize by minimizing variance
- **Negative correlation**: reduces portfolio volatility without reducing expected return
- **Sharpe ratio = (return - risk-free) / std**; maximized at tangency point on efficient frontier
- **Adjusted close price**: always use this for backtesting; adjusts for splits and dividends
- **Stock split**: total value unchanged; price halved (2-for-1), shares doubled
- **Fill forward first**: for missing data — use last known price; fill backward for gaps at start of series
- **Survivorship bias**: including only surviving stocks inflates backtest returns; use complete point-in-time data

---

All 12 sections complete. Good luck on the exam.